



03 — Web Browsers



Introduction

A web browser is one of the most important tools in web development.

It allows users to access websites and web applications on the internet.

Examples of popular web browsers include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari
- Opera

Before learning how to build websites, I want to understand what actually happens **inside a browser** when a webpage is loaded.



What Is a Web Browser?

A web browser is software that allows users to access and interact with resources available on the web.

A browser can:

- Send requests to web servers
- Receive responses
- Download webpages and resources

- Interpret HTML
- Process CSS
- Execute JavaScript
- Display webpages
- Store information such as cookies
- Communicate with web APIs

A simplified process is:

```
User
↓
Web Browser
↓
Internet
↓
Web Server
↓
Internet
↓
Web Browser
↓
Web Page
```

2 What Happens When I Enter a URL?

Suppose I enter:

```
https://example.com
```

into my browser.

The browser has several things to do before displaying the page.

A simplified process is:

```

graph TD
    A[Enter URL] --> B[Browser reads URL]
    B --> C[DNS lookup]
    C --> D[Find server IP address]
    D --> E[Establish connection]
    E --> F[Send HTTP/HTTPS request]
    F --> G[Receive server response]
    G --> H[Process HTML]
    H --> I[Request additional resources]
    I --> J[Process CSS and JavaScript]
    J --> K[Render webpage]

```

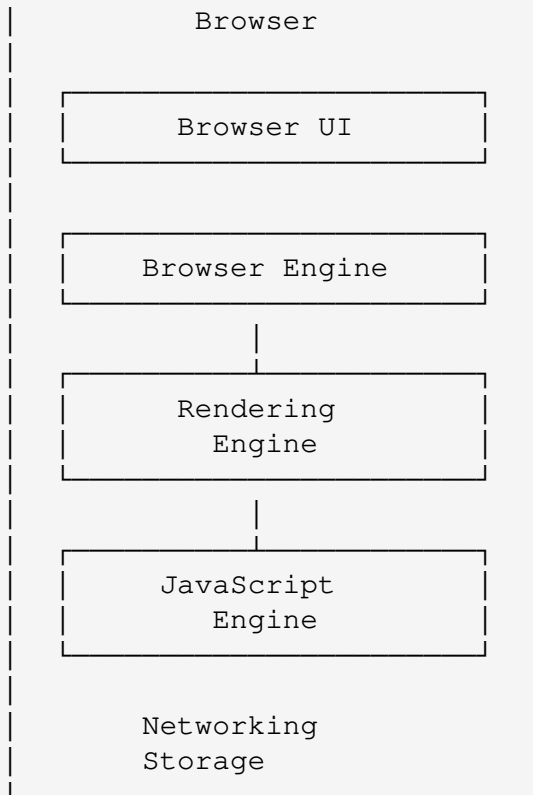
This connects directly to the previous topics I have learned.

3 Browser Components

Modern browsers are complex applications.

A simplified browser architecture contains several important components:





The exact architecture differs between browsers, but these concepts help me understand what is happening.

4 Rendering Engine

One of the most important parts of a browser is the **rendering engine**.

Its job is to take web resources such as HTML and CSS and turn them into the visual webpage I see.

The simplified process is:

```
HTML
↓
DOM
↓
CSS
↓
CSSOM
↓
Render Tree
↓
Layout
↓
Paint
↓
Web Page
```

5📄 HTML Becomes the DOM

When the browser receives HTML, it parses the HTML and creates a structure called the **DOM**.

DOM stands for:

Document Object Model

For example:

```
<h1>Hello World</h1>

<p>Welcome to my website.</p>
```

The browser can represent this as a tree:

```
Document
```

```
|
├── h1
│   └── "Hello World"
└── p
    └── "Welcome to my website."
```

This structure allows JavaScript to interact with webpage elements.

6 CSS Becomes the CSSOM

The browser also processes CSS.

CSS controls how HTML elements should look.

For example:

```
h1 {
  font-size: 40px;
}
```

The browser builds a structure representing the CSS rules.

This is known as the:

CSSOM — CSS Object Model

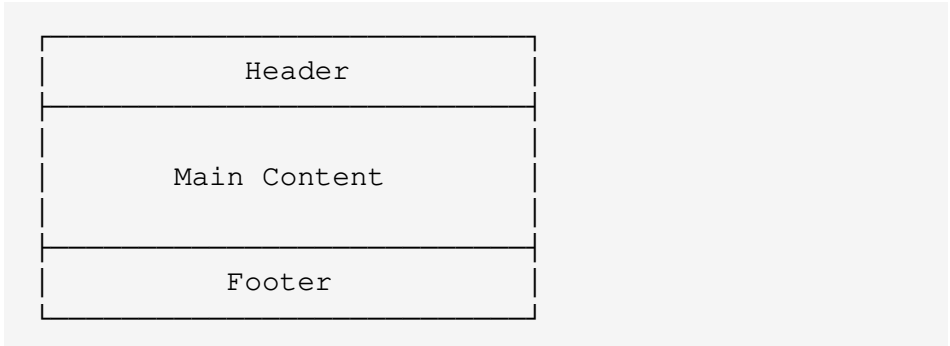
The browser combines information from the DOM and CSSOM to determine how the page should be displayed.

7 Layout

After processing the page structure and styles, the browser determines where elements should appear.

This process is called **layout**.

For example:



The browser calculates things such as:

- Width
- Height
- Position
- Margins
- Padding
- Alignment

8 Painting

After calculating the layout, the browser needs to draw the elements.

This process is called **painting**.

The browser determines things such as:

- Text
- Colors

- Borders
- Shadows
- Images
- Backgrounds

The result is eventually displayed on the screen.

JavaScript Engine

Modern browsers contain a **JavaScript engine**.

Its job is to execute JavaScript code.

Different browsers use different JavaScript engines.

For example:

Browser	JavaScript Engine
Chrome	V8
Edge	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore

For example:

```
console.log("Hello World");
```

The JavaScript engine executes this code.

JavaScript can also interact with the DOM:

```
document.querySelector("h1").textContent = "Hello Mukhtaa
```


This allows a webpage to change dynamically.

Browser Developer Tools

One of the most useful things I can learn as a web developer is how to use **Developer Tools**.

Most modern browsers provide them.

I can usually open them with:

```
F12
```

or:

```
Ctrl + Shift + I
```

Developer Tools can help me inspect:

- HTML
 - CSS
 - JavaScript
 - Network requests
 - Console messages
 - Storage
 - Performance
-

Elements Panel

The **Elements** panel allows me to inspect the HTML structure of a

webpage.

For example:

```
<body>
  <h1>Hello</h1>
  <p>Welcome.</p>
</body>
```

I can inspect the elements and see which CSS rules are being applied.

This will become extremely useful when I start learning CSS.

1.2 Console

The **Console** allows developers to view messages and errors and execute JavaScript.

For example:

```
console.log("Hello from the browser!");
```

The browser displays:

```
Hello from the browser!
```

The console is also useful for debugging JavaScript errors.

1.3 Network Tab

The **Network** tab is especially interesting to me because of my IT and networking background.

It allows developers to see requests made by the browser.

For example:

```
Browser
├── GET index.html
├── GET style.css
├── GET script.js
├── GET image.jpg
└── GET API data
```

I can inspect information such as:

- Request URL
- HTTP method
- Status code
- Response
- Request headers
- Response headers
- Loading time

This helps connect web development with networking.

14 Browser Storage

Browsers can store information locally.

Examples include:

- Cookies
- Local Storage
- Session Storage

- IndexedDB

For example, websites can use cookies to maintain information about a user's session.

This becomes important when learning:

- Authentication
- Login systems
- User preferences
- Web applications

15 Browser Security

Browsers also provide security features.

For example:

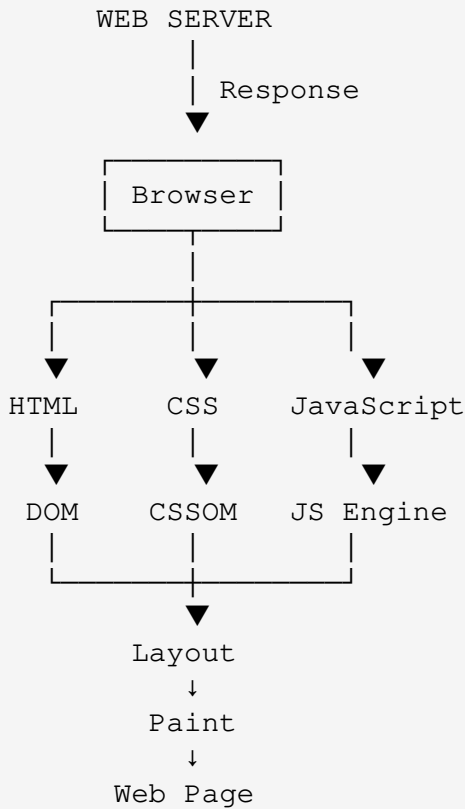
- HTTPS/TLS support
- Same-Origin Policy
- Content Security mechanisms
- Permission controls
- Sandboxing

These mechanisms help protect users and websites from various security threats.

Security will be explored more deeply later in my learning journey.

Putting Everything Together

A simplified view of what happens inside a browser is:



What I Learned

From this topic, I learned that a web browser is much more than a program that simply displays websites.

It acts as a client that communicates with servers, processes web resources, executes JavaScript, manages browser storage, and renders webpages.

The browser takes technologies such as:

HTML
CSS
JavaScript

and turns them into an interactive webpage.

I also learned that Developer Tools allow me to see what is happening behind the scenes.

Key Terms

Term	Meaning
Browser	Software used to access and interact with web content
Rendering Engine	Processes web content and produces the visual page
DOM	Object representation of an HTML document
CSSOM	Object representation of CSS rules
JavaScript Engine	Executes JavaScript
Layout	Determines the size and position of elements
Paint	Draws elements onto the page
Developer Tools	Browser tools used for development and debugging
Console	Tool for JavaScript output and errors
Network Tab	Tool for inspecting network requests
Cookies	Small pieces of data stored by websites
Local Storage	Browser storage for persistent client-side data

? Questions I Want to Explore Next

- How does DNS actually work?
 - How does a browser perform a DNS lookup?
 - What is a domain name?
 - What exactly happens during an HTTP request?
 - How does HTTPS encrypt information?
 - What is web hosting?
 - How does a browser communicate with an API?
 - How does JavaScript interact with the DOM?
-

Progress

Stage: Web Fundamentals

Topic: Web Browsers

Status:  Completed

Previous: [Client-Server Model](#)

Next: Domain Names

A browser doesn't simply display a website — it requests, processes, interprets, and renders it.